Graficación de primitivas 2D
(2 / 2)

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

2026



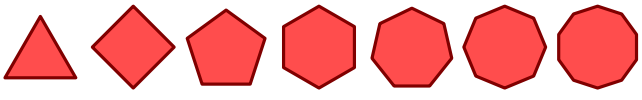
Primitivas de Área vs Primitivas Lineales

- **Primitivas lineales**: Líneas, círculos, arcos
  - Definidas por contornos y perímetros
  - Algoritmos: DDA, Bresenham, punto medio
- **Primitivas de área**: Polígonos, círculos, regiones irregulares
  - Requieren determinar puntos interiores
  - Elegir valor con el que rellenar
  - Algoritmos para relleno de polígonos:
    - Relleno por paridad (*parity fill*)
    - General
  - Algoritmos para relleno de regiones irregulares
    - Relleno de contornos (*Boundary-fill*)
    - Relleno por difusión ó inundación (*Flood-fill*)

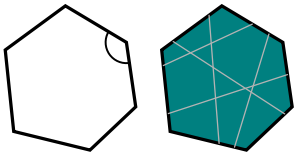


Relleno de polígonos

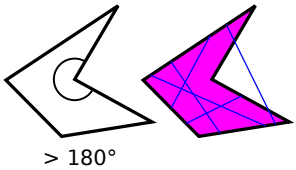
Polígono: *Figura plana que se encuentra definida por tres o más vértices que se encuentran unidos, en secuencia, por medio de segmentos de recta denominados lados*



Polígonos convexos



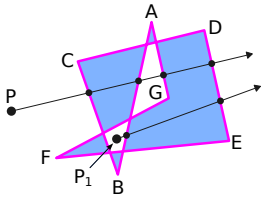
Polígonos cóncavos



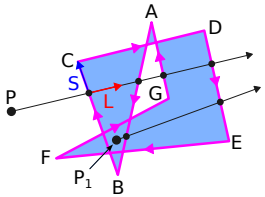
Relleno de polígonos

Determinación Interior/Exterior

- **Regla par-impar (even-odd)**
  - Trazar rayo desde punto arbitrario P hacia infinito
  - Contar intersecciones con contorno
  - Impar = interior, Par = exterior



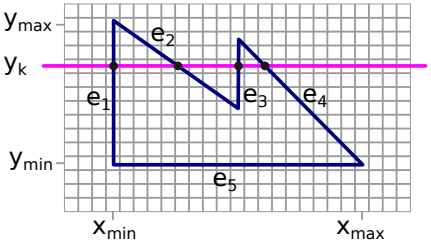
- **Regla nonzero winding**
  - Trazar rayo desde punto arbitrario P hacia infinito
  - *winding number* $= 0$ y se acumula analizando cruces de rayo con lados
  - En cada cruce se evalúa $\widehat{L} \times \widehat{S}$ (producto cruz)
  - Signo de comp. z del producto cruz determina el aporte: $+1$ si $z > 0$, -1 si $z < 0$.
  - Si el winding number final es distinto de cero, P es interior; si es cero, P es exterior.



Algoritmo de relleno por paridad

Para cada polígono hacer

- Identificar extremos del polígono
  - $(x_{min}, x_{max}) \rightarrow$ columnas que ocupa el polígono
  - $(y_{min}, y_{max}) \rightarrow$ scalines a recorrer
- Recorrer cada scanline de izquierda a derecha desde afuera del polígono
- Al iniciar cada scanline fijar un estado como un bit de paridad a 0 (par)
- Al encontrar un borde invertir estado
- Pintar el pixel si el estado es interno (impar)



Algoritmo de relleno por paridad

Ventajas

- Funciona bien para polígonos convexos y cóncavos en general
- Es rápido y se adapta bien para implementar en hardware

Desventajas

- Hay que procesar cada polígono de manera aislada
- No da buenos resultados con figuras que se auto-intersectan

Algoritmo de relleno general

Algoritmo de scanline para polígonos

Tratar de generalizar el algoritmo para todo tipo de polígonos (cóncavos, convexos, auto-intersectados, etc.)

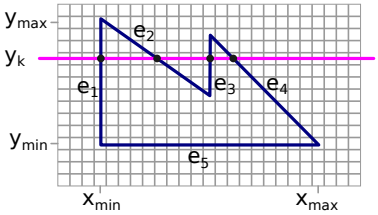
Coherencia: relación, conexión o unión de unas cosas con otras, o aquello que interconecta o mantiene unidas las partes de un todo.

- **Coherencia de la escena**: se hace referencia a las diferentes partes de una escena que podrían brindar una ventaja al momento de procesar el pintado
- **Coherencia espacial**: las primitivas no cambian de pixel a pixel dentro de un intervalo o de una línea de scan a otra
- **Coherencia de línea de scan**: en algunas figuras, tal como los rectángulos las líneas de scan consecutivas son iguales, en algunas figuras generales con líneas iguales se denomina coherencia de Bordes/Lados

Algoritmo scanline para polígonos

Planteo del algoritmo:

1. Recorrer el polígono de abajo hacia arriba
2. Para cada línea de scan ($y = k$) determinar intersecciones con lados del polígono
3. Ordenar intersecciones de manera ascendente según "x" (izquierda a derecha)
4. Recorrer la línea de scan de izquierda a derecha
   - Si la coordenada "x" actual está en la lista de intersecciones invertir estado (lado del polígono)
   - Pintar el pixel si el estado es interno (impar)



Algoritmo scanline para polígonos

Búsqueda de intersecciones con lados del polígono (paso 2)

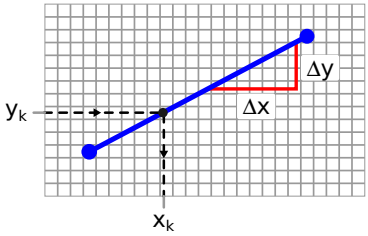
Los lados del polígono son segmentos rectos:

$$y = m \cdot x + b$$

Entonces se pueden encontrar fácilmente las intersecciones de cada línea de scan y_k con los lados de los polígonos resolviendo:

$$x = \frac{y}{m} - \frac{b}{m} \qquad m = \frac{\Delta y}{\Delta x}$$

$$\Rightarrow x_k = \frac{\Delta x}{\Delta y} \cdot y_k - \frac{\Delta x}{\Delta y} \cdot b$$



- Estos valores se ordenan y se almacenan para usarlos en el algoritmo

Algoritmo scanline para polígonos

Manejo de casos especiales (paso 2)

Problema: Scan-line que pasa por un vértice compartido

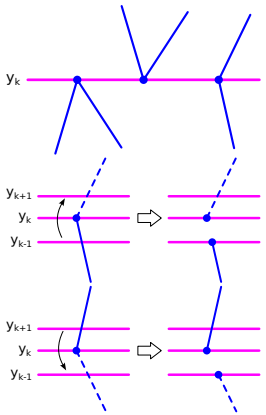
- Puede generar número impar de intersecciones
- Resultado incorrecto en determinación interior/exterior

Solución: Análisis topológico

- **Extremo local**: Bordes en mismo lado → contar como 2 intersecciones
- **Continuación**: Lados en zonas opuestas → contar como 1 intersección

Implementación:

- Recorrer los lados del polígono
- Determinar si lados que comparte un vértice "continúan"
- En este caso acortar lado inferior en 1 unidad



Algoritmo scanline para polígonos

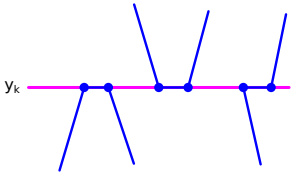
Manejo de casos especiales (paso 2)

Problema: Lados horizontales

- Generan múltiples intersección

Solución: simple

- Ignorar el lado



Optimización del algoritmo de scanline

1. Intersecciones de lados con scanline: Uso de coherencia espacial

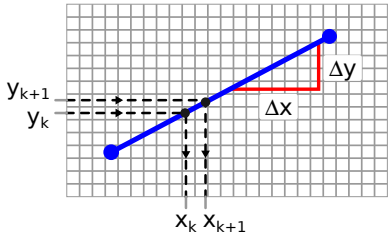
Utilizar *coherencia de bordes* para realizar reconstrucción de lados durante el cálculo de las intersecciones con la línea de scan.

- Se puede calcular la siguiente intersección de un borde a partir de la intersección actual
- Se utiliza algoritmo incremental

$$x_{k+1} = x_k + \frac{1}{m}$$

$$m = \frac{\Delta y}{\Delta x}$$

$$\Rightarrow x_{k+1} = x_k + \frac{\Delta x}{\Delta y}$$



Optimización del algoritmo de scanline

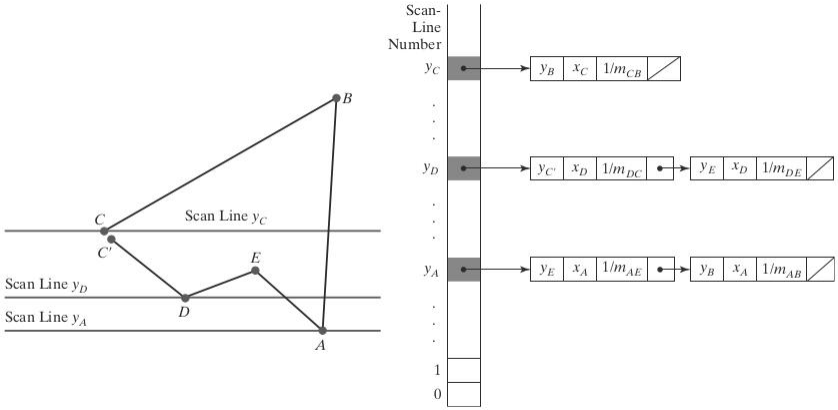
2. Utilizar estructura de datos

Tabla de lados global (Global Edge Table - GET)

- Se utiliza estrucura de datos para usar algoritmo de ordenamiento de casilleros (*bucket sort* o *bin sort*)
- Almacenar lados en una tabla global ordenados por coordenada y_{min}
- Cada entrada de la tabla es un casillero ("bucket")
- En cada casillero ("bucket") se almacenan los datos mínimos para reconstruir los lados que inician o terminan en la línea de scan correspondiente al casillero
- Los datos de reconstrucción se encuentran ordenados por coordenada $x_{min} = f(y_{min})$! de los lados
- Datos para reconstrucción:
  - y_{max}
  - $x_{min} = f(y_{min})$
  - $1/m$

Optimización del algoritmo de scanline

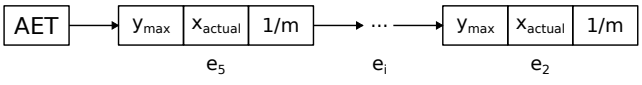
Tabla de lados global (Global Edge Table - GET)

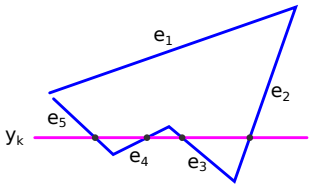


Optimización del algoritmo de scanline

Tabla de lados activos (Active Edge Table - AET)

- Se almacenan los datos de reconstrucción de todos los lados que intersecta la línea de scan activa (y_k)
- Los datos se ordenan de acuerdo al valor de la coordenada "x"





Optimización del algoritmo de scanline

Algoritmo optimizado de scanline para polígonos

- Crear tabla de lados global (GET)
- Inicializar scanline $y_k = y_{min}$ (índice del primer casillero no vacío de GET)
- Inicializar tabla de lados activos (AET) en cero
- Repetir hasta que GET y AET se encuentren vacías
  - Mover lados de GET con $y_{min} = y_k$ a AET
  - Ordenar AET según x (eventualmente AET incuirá valores nuevos y viejos)
  - Pintar pixeles para y_k utilizando pares de coordenadas x de AET
  - Eliminar entradas en AET con $y_{max} = y_k$ (lados que están terminados)
  - Avanzar a la siguiente línea de scan $(y_k + 1)$
  - Actualizar x en las entradas remanentes de AET (sumar 1/m)

Pseudocódigo Scan-Line Optimizado

```
// Datos para reconstruir lados
struct Edge {
    int ymax;       // Coordenada y maxima
    float x;        // Interseccion x actual
    float dx;       // 1/pendiente
};

// Algoritmo principal
void scanLineFill(Polygon poly, Color fillColor) {
    GET = buildSortedEdgeTable(poly);
    AET = NULL;

    for (y = ymin; y <= ymax; y++) {
        // Agregar bordes que inician en y
        addNewEdges(GET[y], AET);

        // Rellenar spans entre pares de intersecciones
        fillSpans(AET, y, fillColor);

        // Remover bordes que terminan en y
        removeCompletedEdges(AET, y);

        // Actualizar intersecciones x para siguiente linea
        updateActiveEdges(AET);
    }
}
```

Algoritmos de relleno de áreas

- Los algoritmos intentan rellenar figuras con bordes curvos o irregulares (no polígonos)
- Usualmente, estos algoritmos son utilizados por artistas gráficos los cuales bosquejan los bordes de una figura y luego seleccionan el color o patrón de relleno desde un menú
- Comienza con un punto conocido dentro de la figura y luego procede a rellenar la figura hacia el exterior del punto
- En paquetes de pintado interactivo, son funciones de pintado como balde de pintura

Conectividad de Píxeles

- **4-conectado**: arriba, abajo, izq, der
- **8-conectado**: incluye diagonales



Algoritmo Relleno de contorno (Boundary Fill)

Concepto:

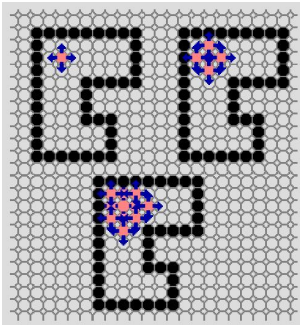
- Partir de punto interior conocido
- Expandir hasta encontrar color de contorno
- Útil para regiones con bordes irregulares

Ventajas:

- Funciona con cualquier forma de contorno
- Implementación intuitiva
- Ideal para aplicaciones interactivas

Desventajas:

- Requiere stack recursivo profundo
- Puede ser lento para áreas grandes
- Problemas si hay píxeles ya coloreados



Pseudocódigo Relleno de contorno (Boundary Fill)

Versión 4-conectada:

```
void boundaryFill4(int x, int y, int fillColor, int borderColor) {
    int interiorColor;

    getPixel(x, y, interiorColor);

    if((interiorColor != borderColor) && (interiorColor != fillColor)) {

        setPixel(x, y, fillColor);

        boundaryFill4(x + 1, y, fillColor, borderColor);
        boundaryFill4(x - 1, y, fillColor, borderColor);
        boundaryFill4(x, y + 1, fillColor, borderColor);
        boundaryFill4(x, y - 1, fillColor, borderColor);
    }
}
```

Optimización por spans:

- Rellenar líneas horizontales completas
- Apilar solo posiciones iniciales de spans
- Reducir uso de memoria del stack

Algoritmo Relleno por difusión (Flood fill)

Diferencia con Relleno de contorno:

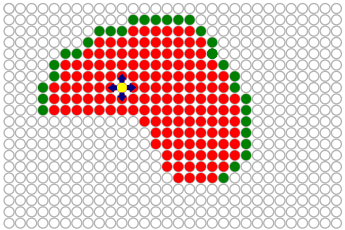
- No busca color de contorno específico
- Reemplaza color interior existente
- Útil para regiones multicolor

Aplicaciones:

- Cambio de color en regiones existentes
- Herramienta "balde de pintura"
- Selección de regiones similares

Casos especiales:

- Regiones con múltiples colores interiores
- Pre-procesamiento para unificar colores



Pseudocódigo Relleno por difusión (Flood fill)

Versión 4-conectada:

```
void floodFill4(int x, int y, int fillColor, int interiorColor) {
    int color;

    getPixel(x, y, color);

    if (color == interiorColor) {
        setPixel(x, y, fillColor);

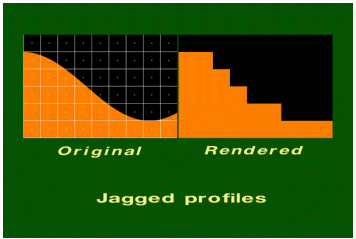
        floodFill4(x + 1, y, fillColor, interiorColor);
        floodFill4(x - 1, y, fillColor, interiorColor);
        floodFill4(x, y + 1, fillColor, interiorColor);
        floodFill4(x, y - 1, fillColor, interiorColor);
    }
}
```

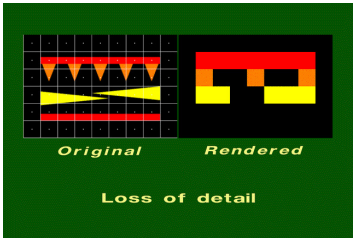
Consideraciones importantes:

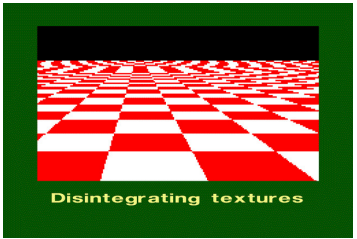
- Verificar que fillColor $\neq$ interiorColor
- Manejar límites del área de dibujo
- Optimización similar por spans horizontales

El problema del "aliasing"

- Los errores causados por el aliasing se denominan "artifacts"
- Artifacts clásicos debidos al aliasing:
  - Perfiles escalonados (jagged)
  - Detalles faltantes o renderizado incorrecto
  - Texturas que se deforman

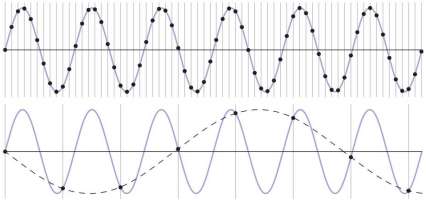






El problema del "aliasing"

- Los algoritmos de primitivas de trama (raster) (Bresenham, punto medio) generan objetos con apariencia *escalonada* (jagged)
- Esta distorsión se debe al proceso de muestreo discreto de modelos continuos
- El aliasing es la pérdida de información causada por submuestreo (undersampling)
- Problema fundamental: representar objetos con parámetros continuos usando píxeles discretos

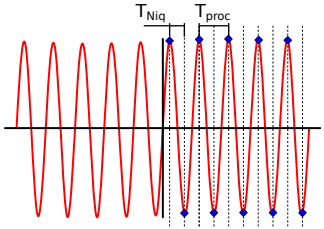


Teoría del muestreo

Para evitar la pérdida de información hay que muestrear con una frecuencia de *al menos* dos veces la frecuencia máxima del proceso (teorema de Nyquist):

$$f_s \geq 2f_{max}$$

$$\Delta x_s \leq \frac{\Delta x_{cycle}}{2}$$



Donde:

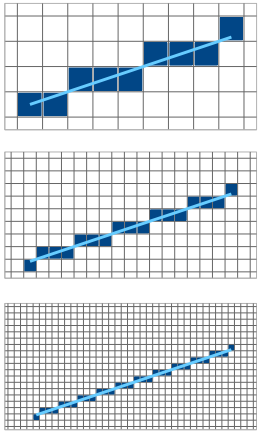
- f_s: frecuencia de muestreo mínima
- f_{max}: frecuencia máxima del objeto
- Δx_s: intervalo de muestreo de Nyquist

Limitación Fundamental

Para representar objetos con parámetros continuos necesitamos intervalos de muestreo arbitrariamente pequeños

Limitaciones de las soluciones por hardware

- **Incrementar resolución:** mejora pero no elimina el problema
- **Limitaciones del frame buffer:**
  - Tamaño máximo limitado por la tecnología
  - Mantener 60+ fps requiere compromisos
  - Costo computacional y de memoria
- **Necesidad de técnicas de software:** antialiasing como solución complementaria



Técnicas de suavizado (anti-aliasing)

1. **Supersampling (Postfiltering)**
   - Muestreo a alta resolución
   - Combinación de resultados
   - Cálculo de intensidades subpíxel
2. **Area Sampling (Prefiltering)**
   - Cálculo de áreas de solapamiento
   - Determinación directa de intensidad
   - Sin cálculos subpíxel
3. **Técnicas de Filtrado**
   - Funciones de peso continuas
   - Integración sobre superficie del píxel
   - Mayor precisión matemática

Criterios de selección - Factores a considerar

- Calidad visual
- Costo computacional
- Memoria requerida
- Disponibilidad en el hardware

Supersampling de líneas rectas

Idea básica:

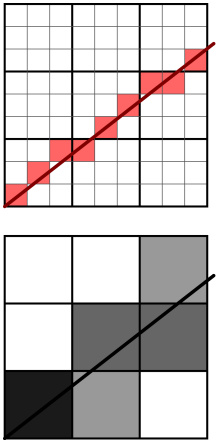
- Renderizar a resolución mayor
- Filtrar y submuestrear

Metodología:

1. Dividir cada píxel en $n \times n$ subpíxeles
2. Aplicar algoritmo de Bresenham en subpíxeles
3. Contar subpíxeles que solapan la línea
4. Establecer intensidad proporcional al conteo

Niveles de intensidad:

- 3×3: hasta 3 niveles
- 4×4: hasta 4 niveles
- 5×5: hasta 5 niveles



Supersampling de líneas de ancho finito

Ventajas del ancho finito:

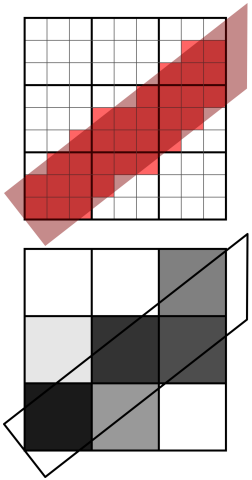
- Todos los subpíxeles pueden contribuir
- Mayor distribución de intensidad
- Se pueden considerar colores de fondo

Cálculo de color con fondo:

$$\text{color}_{pixel} = \frac{n_{linea} \cdot \text{color}_{linea} + n_{fondo} \cdot \text{color}_{fondo}}{n_{total}}$$

Donde:

- n_{linea}: subpixeles dentro de la línea
- n_{fondo}: subpixeles sobre el fondo



Supersampling: máscaras de peso (weighting masks)

Mejora:
Dar más importancia a los subpixeles centrales en la determinación de la intensidad del pixel

Implementación:
Las máscaras se implementan como arrays de pesos que se aplican durante el cálculo de intensidades promedio

Ejemplo máscara 3×3:

1	2	1
2	4	2
1	2	1

$$m_{22} = 4 \quad \text{y} \quad \sum m_{ij} = 16$$

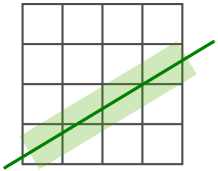
$$\Rightarrow w_{22} = \frac{4}{16} = \frac{1}{4} \quad \text{peso del subpixel central}$$

Area Sampling de líneas

Idea básica: Intensidad proporcional al área de solapamiento píxel-línea

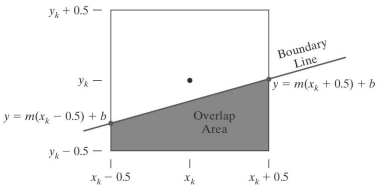
Metodología:

1. Tratar línea como rectángulo de ancho finito
2. Calcular intersección polígono-píxel
3. Determinar porcentaje de cobertura
4. Establecer intensidad proporcional



Ejemplo estimación área:

$$\%\ \text{cobertura} = \frac{S_{trap}}{S_{pixel}} = m \cdot x_k + b - y_k + 1/2$$



Funciones de Filtro

Concepto: Superficie de peso continua sobre el píxel

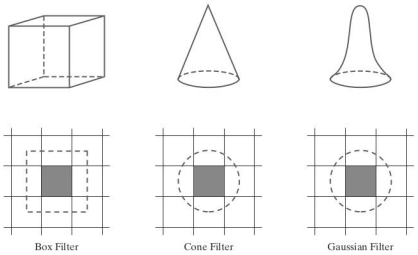
Tipos de filtros comunes:

- **Box Filter:** peso uniforme
- **Cone Filter:** peso decreciente radial
- **Gaussian Filter:** distribución gaussiana

Aplicación:

$$I_{pixel} = \iint_{pixel} f(x,y) \cdot w(x,y)\, dx\, dy$$

Donde $f(x,y)$ es la función objeto y $w(x,y)$ el filtro



Bibliografía recomendada

- HEARN, D. & BAKER, M. P.. *Computer Graphics with OpenGL*. 4ta ed. Essex (UK): Pearson Education Limited, 2014. cap. 5 Implementation algorithms for graphics primitives and attributes, p. 53-64.
- HEARN, D. & BAKER, M. P.. *Computer Graphics with OpenGL*. 4ta ed. Essex (UK): Pearson Education Limited, 2014. cap. 5 Implementation algorithms for graphics primitives and attributes, p. 165-189.

¿¿¿ PREGUNTAS ???